Ensemble Learning: Introduction and Fundamentals

Simon Liu

Oct 15, 2026

Self Introduction

Roles

- Chief Data and AI Officer, TrustDecision.
- Adjunct Associate Professor at NTU.
- Teaching both EE6405 (with Dr. Supraja) and EE6406.

Past Experience

- Senior Vice President, Head of Data Science (risk), Lazada.
- Director in Data Science, Scotiabank, Canada.

Education

- Ph.D. in Electrical and Computer Engineering from the University of Toronto.
- Learned deep learning from Prof. Geoffrey Hinton.

Teaching in EE6406

- Week 9 - 13.
- Ensemble Learning, Reinforcement Learning, Industry applications.

Teaching Style

Regular Week Breakdown

- Lecture Part 1
- 15 mins break
- Lecture Part 2
- Practical - Sample codes and/or review
- Summary
- Q & A

Weekly Schedule

1. **Week 9**
   - Introduction to Ensemble Learning, Decisions Trees, Bagging
2. **Week 10**
   - Random Forest, Boosting, AdaBoost
3. **Week 11**
   - Gradient-Boosted Trees, XGBoost, LightGBM
4. **Week 12**
   - Introduction to Reinforcement Learning
   - Quiz 3
5. **Week 13**
   - Industrial Applications
   - Review

Week 9 - Content

Linear Models

Linear models are fundamental in machine learning but face limitations in complex data scenarios.

- **What is a Linear Model?**
  - Predicts outcomes based on a linear combination of input features.
  - Equation: $y = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$
- **Examples of Linear Models:**
  - Linear Regression (for continuous outputs)
  - Logistic Regression (for classification tasks)
  - Support Vector Machines (with linear kernels)
- **Why Linear Models?**
  - Simple and interpretable
  - Efficient for high-dimensional data when feature relationships are linear

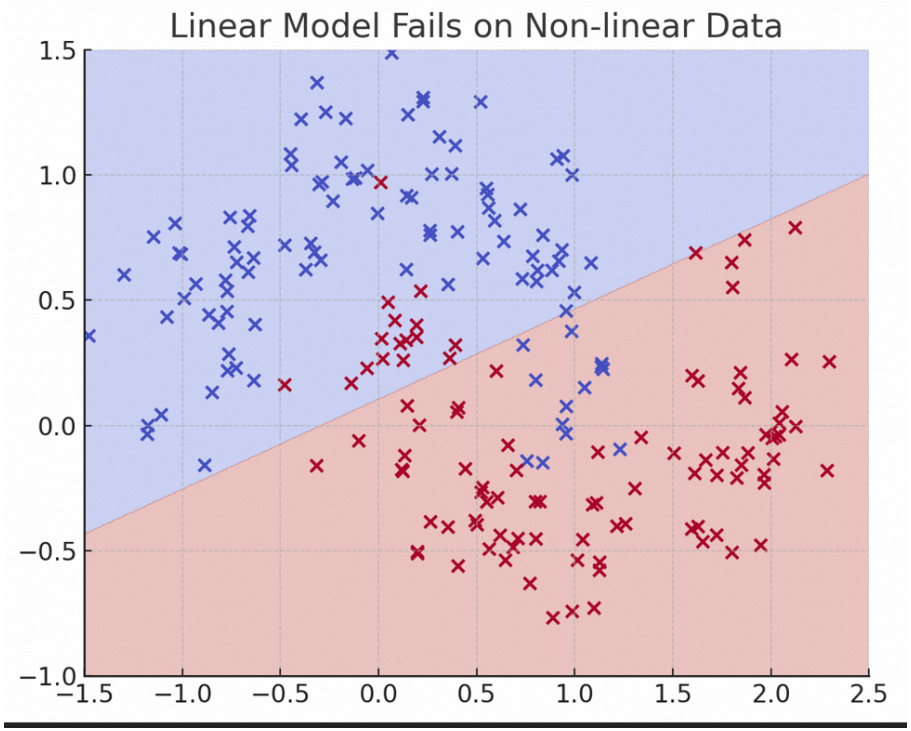
Limitations of Linear Models

Linear models struggle in real-world scenarios where data relationships are complex.

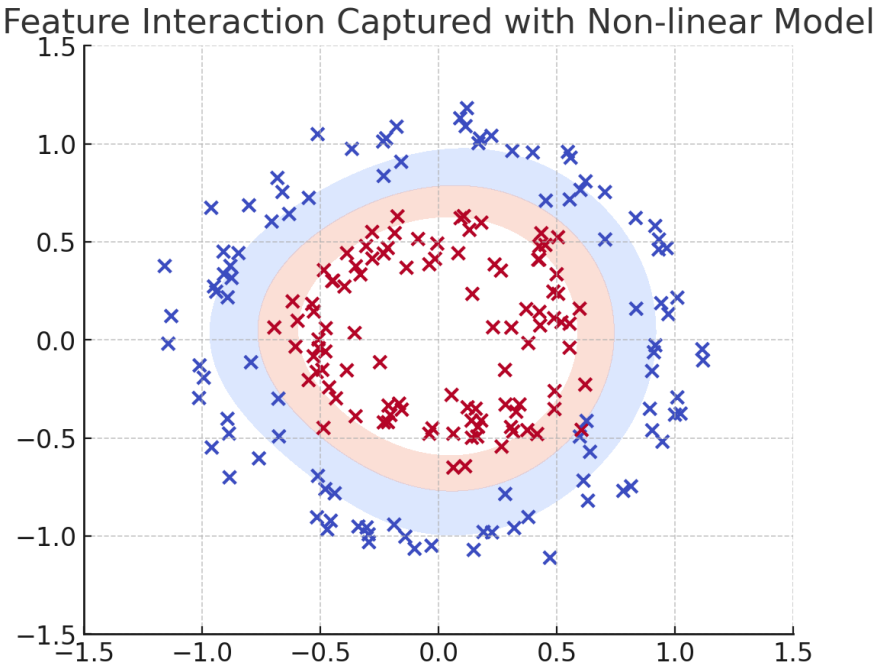
- **Non-Linear Relationships:**
  - Cannot capture complex decision boundaries
  - Example: XOR problem cannot be solved with a linear model
- **Feature Interactions:**
  - Assumes features contribute independently
  - Struggles to capture complex interactions
- **Outlier Sensitivity:**
  - Prone to extreme values, impacting model performance
- **Overfitting/Underfitting:**
  - Simple linear models may underfit
  - Complex models can overfit without proper regularization
- **Limited Flexibility:**
  - Struggles in multi-modal data where classes overlap

Illustrative Example: Linear Model Limitations

Linear Model Fails on Non-linear Data



Non-linear Model Captures Complex Pattern



Key Takeaway: Linear models excel in simple cases but struggle with complex patterns, motivating the need for ensemble methods.

Trade-off Between Interpretability and Predictive Power

There is a trade-off between model interpretability and predictive power.

Aspect	Linear Models	Ensemble Models	Deep Learning
Interpretability	High	Moderate	Low
Predictive Power	Low	High	Very High
Overfitting Risk	Low	Moderate	High
Training Speed	Fast	Moderate	Slow
Non-linear Data	Poor	Excellent	Excellent
Data Needs	Low	Moderate	High

Key Insight: Ensemble models strike the best balance — powerful yet interpretable — ideal for real-world applications.

Introduction Ensemble Learning

Introduction to Ensemble Learning

Definition

- Ensemble learning combines multiple models (often called weak learners) to build a stronger predictive model.
- Inspired by the "wisdom of the crowd" — collective decisions often outperform individual ones.

Analogy

- Imagine a jury deciding a case — while individual jurors may make errors, a collective decision through majority vote often leads to better outcomes.

Condorcet Jury Theorem

"If each juror has a better than random chance of being correct, adding more jurors increases the probability that the majority decision is correct."

Why Ensemble Learning?

Why Ensemble Learning?

- Individual models can struggle with:
  - **Bias** — Errors from overly simplistic assumptions.
  - **Variance** — Errors from excessive sensitivity to noise.
- Ensemble learning mitigates these issues by combining multiple models to improve accuracy and stability.

Key Advantages

- **Robustness** — Reduces overfitting by averaging multiple models.
- **Improved Accuracy** — Aggregating diverse models enhances performance.
- **Stability** — Less sensitive to data noise or outliers.
- **Versatility** — Effective for both classification and regression tasks.
- **Reduced Model Dependency** — Often outperforms finely-tuned single models with minimal hyperparameter tuning.

Framework for Combining Weak Learners

General Aggregation Strategy

- Ensemble models combine multiple weak learners into a stronger predictor.
- Given weak learners $f_m(x)$ for $m = 1, 2, \ldots, M$, the combined model is:

$$F(x) = \Phi\left(\sum_{m=1}^{M} \alpha_m f_m(x)\right)$$

Components

- Φ: Activation function translating the combined predictions into the final output.
- α_m: Weight for the m-th weak learner.
  - **In Bagging:** $\alpha_m = \frac{1}{M}$ (equal weights).
  - **In Boosting:** α_m is adjusted based on learner performance.

Aggregation for Classification and Regression

Classification

- For binary classification:

$$\Phi(z) = \text{sign}(z)$$

- For multi-class classification:

$$\Phi(z) = \arg\max(z_k)$$

where z_k represents the aggregated score for class k.

Regression

- For regression tasks:

$$\Phi(z) = z$$

- The combined model becomes:

$$F(x) = \sum_{m=1}^{M} \alpha_m f_m(x)$$

Understanding Bias and Variance

Bias

- Error introduced by approximating the true data pattern with a simpler model.
- **High Bias** — The model is too simplistic and underfits the data.
- **Mathematical Definition:**

$$\text{Bias}[\hat{f}(x)] = \mathbb{E}[\hat{f}(x)] - f(x)$$

Variance

- Sensitivity of the model to small changes in the training data.
- **High Variance** — The model is too complex and overfits the data.
- **Mathematical Definition:**

$$\text{Var}[\hat{f}(x)] = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$

Key Insight: Ensemble methods aim to reduce variance (e.g., Bagging) or reduce bias (e.g., Boosting) while balancing both.

Prediction Error Decomposition and Bias-Variance Trade-off

Prediction Error Decomposition

- The total prediction error can be decomposed as:

$$\mathbb{E}[(\hat{f}(x) - f(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

- Where:
  - $\hat{f}(x)$ = Model's predicted output
  - $f(x)$ = True function
  - σ^2 = Irreducible error (random noise inherent to the data)

Bias-Variance Trade-off

- Increasing model complexity generally:
  - **Decreases Bias** — Better fits the training data.
  - **Increases Variance** — Becomes overly sensitive to noise.
- **Goal:** Achieve the optimal balance between bias and variance for minimal total error.

Choosing Weak Learners: Design Choices

- **Simplicity**
  - Weak learners should be simple models that capture basic patterns in the data.
  - Avoid overly complex models, which risk high variance and limit ensemble diversity.
- **Diversity**
  - Models with different structures, features, or data subsets enhance ensemble robustness.
- **Moderate Performance**
  - Weak learners are often deliberately limited in their predictive power (e.g., shallow trees, simple rules).
  - Overly strong learners may dominate the ensemble, reducing the benefits of combining multiple models.

Common choice: Decision Trees

Decision Trees

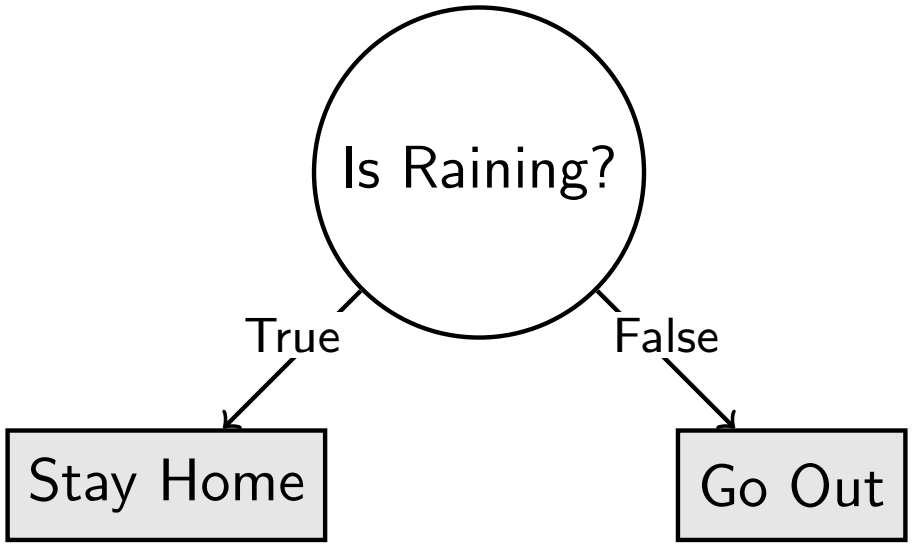
Why Decision Trees Are a Common Choice

- **Flexibility**
  - Decision trees can handle both numerical and categorical data.
- **Fast Training**
  - Shallow trees (e.g., tree stumps) are quick to train, ideal for iterative methods like boosting.
- **Interpretable Rules**
  - Decision tree splits provide intuitive decision rules, aiding model transparency.
- **Natural Diversity**
  - Decision trees are prone to high variance, making them ideal for bagging, which reduces variance through averaging.
- **Adaptive Splits**
  - Trees naturally adapt to non-linear patterns, making them powerful weak learners when combined in ensembles.

Simple Binary Decision Tree

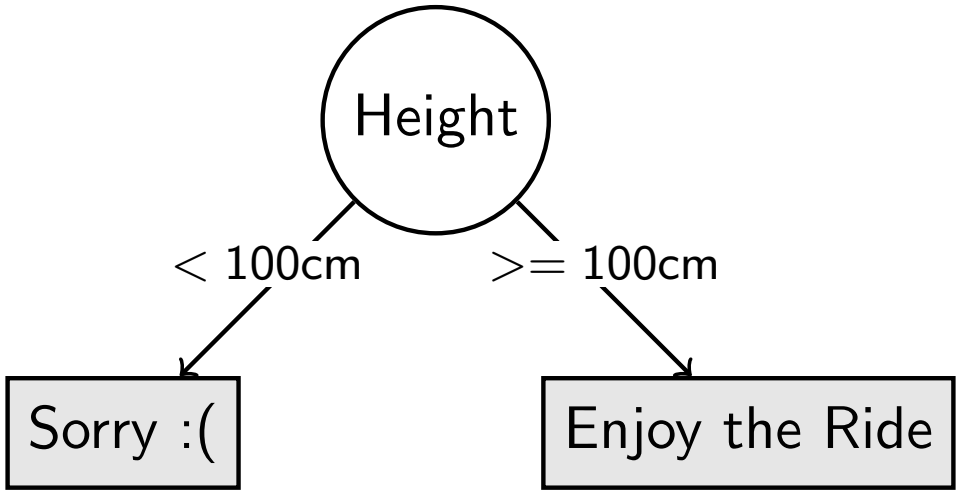
Introduction

- This is an example of the simplest form of a decision tree.
- The tree uses a **single binary feature** — `Is Raining?`.
- Binary decision trees are common in straightforward "Yes/No" decision problems.



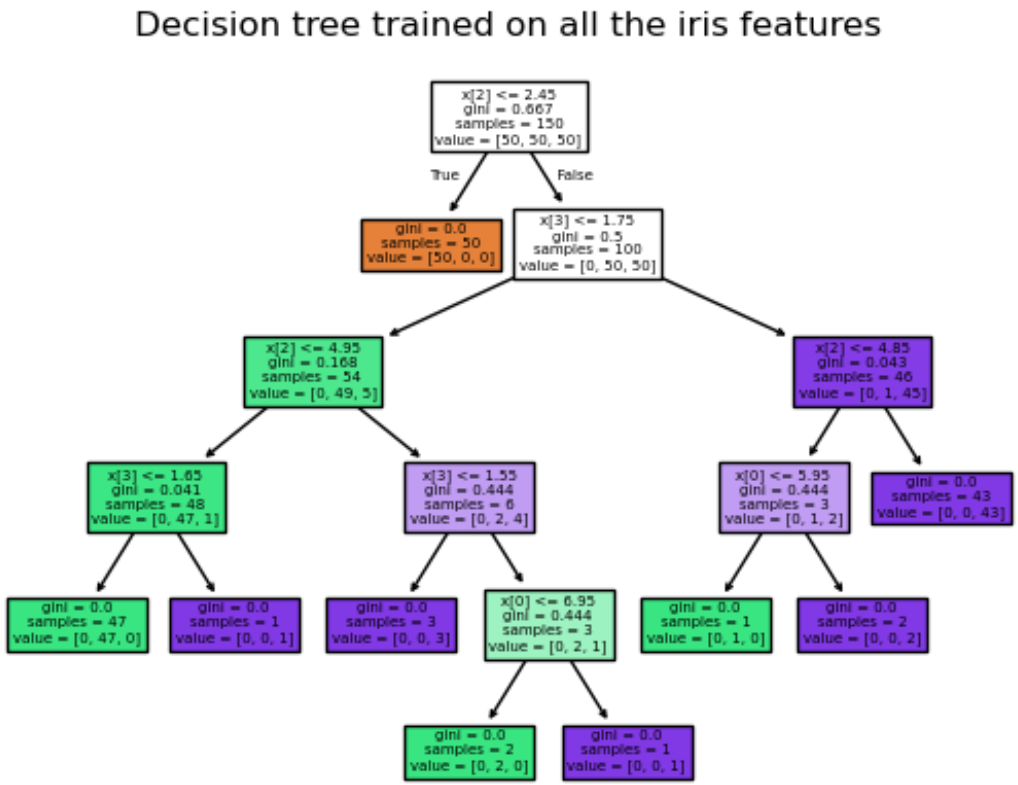
Simple Numeric Decision Tree

- This is an example of a simple a decision tree with numeric boundaries



Decision Tree Example: Iris Dataset

- Decision trees can grow complex when dealing with multiple features and intricate decision boundaries.
- As complexity increases, manual visualization becomes difficult.
- Typically, we use software like `scikit-learn` to build and visualize complex decision trees efficiently.



Building an Optimal Decision Tree (Part 1)

Objective

- Goal: Find the **smallest decision tree** that correctly classifies all data points.
- Optimal trees balance simplicity with accuracy to minimize overfitting.

Why is this Difficult?

- Finding the optimal decision tree is an **NP-hard** problem.
- The number of possible tree configurations grows exponentially with data size.

Building an Optimal Decision Tree (Part 2)

Practical Approach

- Instead of finding the perfect tree, we build the tree **greedily**.
- Each step selects the best split to **maximize homogeneity** in the child nodes.

Maximizing Homogeneity

- Ensures child nodes are as pure as possible (data points of the same class).
- Common measures for evaluating splits:
  - **Gini impurity**
  - **Entropy**
  - **Variance reduction**

Split Criterion for Regression

Objective in Regression Splits

- Goal: Minimize the total variance within child nodes after a split.

Variance Definition

- For a node $\mathcal{N}$ with n data points:

$$\text{Var}(\mathcal{N}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2$$

where:

- y_i = Target value for observation i
- $\bar{y}$ = Mean of the target values in the node

Variance Reduction Formula (Split Criterion)

- The reduction in variance from a split is:

$$\Delta\text{Var} = \text{Var}(\mathcal{N}) - \left(\frac{n_L}{n}\text{Var}(\mathcal{N}_L) + \frac{n_R}{n}\text{Var}(\mathcal{N}_R)\right)$$

- Maximizing ΔVar ensures child nodes are as homogeneous as possible.

Split Criterion for Classification — Gini Impurity

Gini Impurity Definition

- Measures how often a randomly chosen element would be incorrectly classified.
- For a node $\mathcal{N}$ with K classes:

$$\text{Gini}(\mathcal{N}) = 1 - \sum_{k=1}^{K} p_k^2$$

where p_k is the proportion of samples in node $\mathcal{N}$ belonging to class k.

Gini Reduction (Split Criterion)

- The reduction in Gini impurity from a split is:

$$\Delta\text{Gini} = \text{Gini}(\mathcal{N}) - \left(\frac{n_L}{n} \text{Gini}(\mathcal{N}_L) + \frac{n_R}{n} \text{Gini}(\mathcal{N}_R) \right)$$

- Maximizing ΔGini ensures child nodes are as pure as possible.

Split Criterion for Classification — Entropy

Entropy Definition

- Measures the uncertainty (disorder) within a node.
- For a node $\mathcal{N}$ with K classes:

$$\text{Entropy}(\mathcal{N}) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

where p_k is the proportion of samples in node $\mathcal{N}$ belonging to class k.

Entropy Reduction (Information Gain)

- The reduction in entropy from a split is:

$$\Delta\text{Entropy} = \text{Entropy}(\mathcal{N}) - \left(\frac{n_L}{n}\text{Entropy}(\mathcal{N}_L) + \frac{n_R}{n}\text{Entropy}(\mathcal{N}_R)\right)$$

- Maximizing ΔEntropy ensures child nodes are more organized (less disorder) and homogeneous.

Recursive Algorithm for Building a Decision Tree (Part 1)

Function: BuildTree(D, depth)

1. **Stopping Condition:**
   - If all samples in D belong to the same class, or maximum depth is reached:
     - **Return a leaf node:**
     - Majority class (classification)
     - Average value (regression)
2. **Find Best Split:**
   - Evaluate all features and split points.
   - Choose the split that maximizes homogeneity:
     - **For Regression:** Minimize variance within child nodes.
     - **For Classification:** Minimize:
       - Gini impurity
       - Entropy

Recursive Algorithm for Building a Decision Tree (Part 2)

3. **Split the Data:**
   - Partition dataset D into:
     - D_L = Data satisfying the split condition
     - D_R = Remaining data
4. **Recursively Build the Tree:**
   - Left Child = **BuildTree(D_L, depth + 1)**
   - Right Child = **BuildTree(D_R, depth + 1)**
5. **Return Decision Node:**
   - The chosen split condition
   - Left and right child pointers

Additional Stopping Criteria

Additional Stopping Criteria

- **Minimum Samples Split:** Stop splitting if node has fewer than a set number of samples.
- **Minimum Samples per Leaf:** Reject splits if any child node has too few samples.
- **Maximum Leaf Nodes:** Limit the total number of leaf nodes to control complexity.
- **Maximum Impurity Decrease:** Split only if impurity reduction exceeds a set threshold.

Purpose:

- Improves model **efficiency**.
- Enhances **robustness**.
- Reduces risk of **overfitting**.

Bias and Variance in Decision Trees

Bias:

- Decision trees generally have **low bias**.
- They make minimal assumptions about the data and capture complex patterns when grown deep.
- However, **shallow trees** may exhibit **high bias**, failing to capture key trends (underfitting).

Variance:

- Decision trees tend to have **high variance**.
- Small changes in training data can drastically alter the tree structure.
- **Deep trees** risk **overfitting**, memorizing noise rather than learning true patterns.

Summary:

- **Shallow Trees $\rightarrow$ High Bias, Low Variance** (Underfitting)
- **Deep Trees $\rightarrow$ Low Bias, High Variance** (Overfitting)

Advantages of Decision Trees

Advantages:

- **Interpretability:** Easy to visualize and understand.
- **Flexibility:** Handles both categorical and numerical data.
- **Non-parametric Nature:** No assumptions about data distribution.
- **Feature Importance:** Identifies key features naturally.
- **Robust to Irrelevant Features:** Ignores unimportant features.
- **Fast Training and Prediction:** Efficient on large datasets.

Limitations of Decision Trees

Limitations:

- **Overfitting:** Deep trees may memorize training data.
- **High Variance:** Sensitive to small data changes.
- **Limited Expressiveness:** Struggles to model complex patterns.
- **Biased Splitting:** Favors features with many distinct values.
- **Suboptimal for Imbalanced Data:** May produce biased results.
- **Lack of Robustness to Noise:** May create branches fitting noise.

Break

Bagging

Introduction to Bagging

What is Bagging?

- Short for **Bootstrap Aggregation**.
- An ensemble method designed to improve model stability and accuracy by reducing **variance**.

Key Idea:

- Train multiple models independently on **random subsets** of the training data.
- Combine their predictions via:
  - **Averaging** (for regression)
  - **Majority voting** (for classification)

Why Bagging Works:

- Averaging multiple models smooths noise and reduces overfitting.
- Particularly effective for **high-variance models** like decision trees.

Example Algorithm: Random Forest

The Idea of Bootstrapping

What is Bootstrapping?

- Introduced by **Bradley Efron** in **1979**.
- A method that creates multiple datasets by sampling **with replacement**.

Key Idea:

- Each bootstrap sample may contain **duplicate entries** while some points may be excluded.
- This randomness introduces **diversity** in the training data.

Purpose in Bagging:

- Bootstrapping creates **varied datasets**, allowing multiple models to learn different data aspects.
- Combining these models reduces **variance** and improves model stability.

Why Important? Useful for estimating uncertainty and improving robustness, especially with limited data.

Bootstrapping Techniques (Part 1)

1. Simple Bootstrap

- Randomly sample **with replacement** until the sample size equals the original dataset.
- Each observation may appear multiple times or not at all.
- For dataset $D = \{x_1, x_2, \ldots, x_N\}$:

$$D_i = \{x_1^*, x_2^*, \ldots, x_N^*\} \quad \text{where} \quad x_j^* \sim D$$

2. Stratified Bootstrap

- Ensures each class is represented proportionally to the original dataset.
- For class k, sample points with probability:

$$p_k = \frac{N_k}{N}$$

Bootstrapping Techniques (Part 2)

1. Poisson Bootstrapping

- For each observation x_i, draw a random count c_i from a **Poisson(1)** distribution.
- The bootstrap sample is constructed as:

$$D_i = \{x_1, \ldots, x_1, x_2, \ldots, x_2, \ldots, x_N\}$$

where each observation x_i appears c_i **times**.

2. Bayesian Bootstrapping

- Each observation receives a **random weight** drawn from an **Exponential(1)** or **Uniform(0,1)** distribution.
- The bootstrap sample is:

$$\tilde{p}_i = \frac{w_i}{\sum_{j=1}^{N} w_j} \quad \text{where} \quad w_i \sim \text{Exponential}(1)$$

Aggregation in Bagging (Part 1)

Training Weak Models

- Each weak model is trained on a unique **bootstrap sample** drawn **with replacement**.
- Different bootstrap samples lead models to learn different patterns, enhancing diversity.

Aggregation of Predictions (Regression)

- After training, model outputs are combined to improve performance.

$$\hat{y} = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x)$$

where:

- M = Number of weak models.
- $\hat{f}_m(x)$ = Prediction of the m-th weak model.

Aggregation in Bagging (Part 2)

Aggregation of Predictions (Classification - Majority Voting)

- For classification tasks:

$$\hat{y} = \arg\max_k \sum_{m=1}^{M} \mathbb{I}(\hat{f}_m(x) = k)$$

where:

- M = Number of weak models.
- $\hat{f}_m(x)$ = Prediction of the m-th weak model.
- $\mathbb{I}(\cdot)$ = Indicator function that returns 1 if the condition is true, otherwise 0.

Key Properties of Aggregation:

- **Reduces Variance:** Combining multiple models smooths noise.
- **Improves Stability:** Diverse models capture different data characteristics.
- **Bias Remains Unchanged:** Bias reduction is minimal since weak models are typically low-bias learners.

Other Aggregation Techniques

1. Weighted Averaging

- Assigns **different weights** to models based on performance.
- More accurate models receive **higher weights**.
- For regression:

$$\hat{y} = \sum_{m=1}^{M} \alpha_m \hat{f}_m(x)$$

  where $\alpha_m \geq 0$ and $\sum_{m=1}^{M} \alpha_m = 1$.
- Weights are often optimized via **cross-validation**.

2. Differential Weighting

- Weights may vary by **input region** or characteristics.
- For observation x_i:

$$\hat{y}_i = \sum_{m=1}^{M} \alpha_m(x_i) \hat{f}_m(x_i)$$

- Useful in **heterogeneous data** where models specialize in different regions.

Stacking (Stacked Generalization)

What is Stacking?

- An advanced ensemble technique that uses a **meta-learner** to combine weak model outputs.

Formulation:

- Suppose we have M weak models, each producing predictions:

$$Z = \begin{bmatrix} \hat{f}_1(x_1) & \hat{f}_2(x_1) & \cdots & \hat{f}_M(x_1) \\ \hat{f}_1(x_2) & \hat{f}_2(x_2) & \cdots & \hat{f}_M(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \hat{f}_1(x_N) & \hat{f}_2(x_N) & \cdots & \hat{f}_M(x_N) \end{bmatrix}$$

- The meta-model predicts the final output:

$$\hat{y} = g(Z) = g(\hat{f}_1(x), \hat{f}_2(x), \ldots, \hat{f}_M(x))$$

where $g(\cdot)$ is the meta-model (e.g., logistic regression).

Example:

- Combine decision trees, SVMs, and neural networks as base models.
- Use a **logistic regression** model as the meta-learner.

Variance Reduction Through Averaging (Part 1)

- Suppose we have M independent models, each predicting:

$$\hat{f}_m(x) = f(x) + \epsilon_m$$

- Where:
  - $f(x)$ = True underlying function.
  - ϵ_m = Zero-mean noise with variance σ^2.
- Noise terms are assumed to be **independent and identically distributed (i.i.d.)**.

Averaged Model Prediction

- The bagging ensemble averages the predictions:

$$\hat{f}_{\text{avg}}(x) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x)$$

- Since $\mathbb{E}[\epsilon_m] = 0$, the expected value is:

$$\mathbb{E}[\hat{f}_{\text{avg}}(x)] = f(x)$$

- The averaged model is an **unbiased estimator** of the true function.

Variance Reduction Through Averaging (Part 2)

Variance of the Averaged Model

- Using the variance properties:

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \text{Var}\left(\frac{1}{M}\sum_{m=1}^{M}\hat{f}_m(x)\right)$$

- Since noise terms are independent:

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \frac{1}{M^2}\sum_{m=1}^{M}\text{Var}(\hat{f}_m(x)) = \frac{1}{M^2}\sum_{m=1}^{M}\sigma^2 = \frac{\sigma^2}{M}$$

- The variance of the averaged model decreases proportionally to $\frac{1}{M}$.

Key Conclusion:

- Bagging effectively reduces variance while retaining the same bias.
- This improved stability makes bagging particularly powerful for **high-variance models** like decision trees.

Correlation in Bagging and Random Forests

Assumption of Independence

- The previous variance reduction result assumes that models are **uncorrelated**.
- In reality, models trained on similar data often have **correlated errors**, limiting the effectiveness of averaging.

How Bootstrapping Reduces Correlation

- Bootstrapping introduces randomness:
  - Different samples may include distinct points.
  - Models are forced to focus on different data patterns.
- This reduces model correlation and improves ensemble diversity.

Random Forest

- **Random Forests** further reduce correlation by randomly selecting features at each split. *(To be discussed in detail next week.)*

Variance Reduction for Correlated Models (Part 1)

- Suppose we have M models, each predicting:

$$\hat{f}_m(x) = f(x) + \epsilon_m$$

- Where:
  - $f(x)$ = True underlying function.
  - ϵ_m = Zero-mean noise with variance σ^2.
- Noise terms are no longer independent — they now have **correlation**.

Averaged Model Prediction

- The ensemble's prediction is:

$$\hat{f}_{\text{avg}}(x) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x)$$

- Variance of the averaged model:

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \frac{1}{M^2} \sum_{m=1}^{M} \text{Var}(\hat{f}_m(x)) + \frac{2}{M^2} \sum_{1 \le i < j \le M} \text{Cov}(\hat{f}_i(x), \hat{f}_j(x))$$

Variance Reduction for Correlated Models (Part 2)

Expanding the Terms

- Since $\text{Var}(\hat{f}_m(x)) = \sigma^2$ and assuming each pair has correlation ρ,

$$\text{Cov}(\hat{f}_i(x), \hat{f}_j(x)) = \rho\sigma^2$$

- Final variance expression:

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \frac{\sigma^2}{M} + \frac{M(M-1)}{M^2}\rho\sigma^2$$

- Simplifying:

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \sigma^2 \left(\frac{1-\rho}{M} + \rho \right)$$

Key Observations:

- If models are **uncorrelated** ($\rho = 0$), variance reduces by $\frac{1}{M}$.
- If models are **fully correlated** ($\rho = 1$), averaging provides **no variance reduction**.
- Effective ensemble methods aim to reduce ρ through techniques like **bootstrapping** and **Random Forests**.

Impact on Bias in Bagging (Part 1)

Bias in Bagging

- Bagging is primarily designed to reduce **variance**, not bias.
- Since bagging combines multiple models without modifying their internal structure, the **bias** of the ensemble remains similar to the bias of individual models.

Why Bias Remains Unchanged

- Recall that each weak model predicts:

$$\hat{f}_m(x) = f(x) + \epsilon_m$$

- Averaging across models:

$$\hat{f}_{\text{avg}}(x) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x)$$

Impact on Bias in Bagging (Part 2)

Taking the Expectation

- Taking the expectation:

$$\mathbb{E}[\hat{f}_{\text{avg}}(x)] = \mathbb{E}\left[\frac{1}{M}\sum_{m=1}^{M}\hat{f}_m(x)\right]$$

- By the **linearity of expectation**:

$$\mathbb{E}[\hat{f}_{\text{avg}}(x)] = \frac{1}{M}\sum_{m=1}^{M}\mathbb{E}[\hat{f}_m(x)]$$

Expanding the Expectation

- From our earlier setup,

$$\mathbb{E}[\hat{f}_m(x)] = \mathbb{E}[f(x) + \epsilon_m]$$

- Since $f(x)$ is deterministic (not random) and $\mathbb{E}[\epsilon_m] = 0$,

$$\mathbb{E}[\hat{f}_m(x)] = f(x) + \mathbb{E}[\epsilon_m] = f(x) + 0 = f(x)$$

Impact on Bias in Bagging (Part 3)

- Thus we have:

$$\mathbb{E}[\hat{f}_{\text{avg}}(x)] = f(x)$$

- The ensemble is an **unbiased estimator** if the individual models are unbiased.

Key Implications

- Bagging excels with **high-variance** models like decision trees.
- It provides minimal improvement for **high-bias** models like linear regression unless combined with additional complexity.
- For improving bias, consider methods like **Boosting**.

Coding Examples

Summary

Advantages of Ensemble Learning and Decision Trees

Why Use Ensemble Learning?

- Combines multiple models to improve **accuracy**, **stability**, and **robustness**.
- Reduces the risk of overfitting by leveraging model diversity.
- Effective in reducing both **variance** (e.g., Bagging) and **bias** (e.g., Boosting).

Advantages of Decision Trees in Ensembles

- **Flexible** — Handles both numerical and categorical data.
- **Non-parametric** — No assumptions about data distribution.
- **Interpretable** — Easy to visualize and explain.
- **High Variance** — Naturally suited for variance-reducing methods like Bagging.
- **Fast Training** — Ideal for constructing multiple trees efficiently.

Bagging - Concept and Mechanism

What is Bagging?

- Short for **Bootstrap Aggregation** — an ensemble method designed to improve model stability and accuracy.
- Bagging reduces **variance** by averaging predictions from multiple models trained on **bootstrap samples**.

Bootstrap Sampling

- Creates multiple datasets by sampling **with replacement**.
- Each bootstrap sample is the same size as the original dataset but may contain **duplicate entries**.
- Introduces randomness to improve model diversity.

Aggregation Techniques

- For **Regression** $\rightarrow$ Use **averaging**:
- For **Classification** $\rightarrow$ Use **majority voting**:

Bagging - Impact on Bias and Variance

Variance Reduction

- Bagging reduces variance by averaging model predictions.
- Variance reduction is proportional to $\frac{1}{M}$ for uncorrelated models:

$$\text{Var}(\hat{f}_{\text{avg}}) = \sigma^2 \left(\frac{1-\rho}{M} + \rho \right)$$

Managing Correlation

- Bootstrapping reduces correlation by providing diverse training data.
- **Random Forests** further reduce correlation by randomly selecting features at each split. *(To be discussed next week.)*

Impact on Bias

- Bagging has **minimal impact on bias**.
- Bias reduction is possible by using more complex weak models or combining bagging with other methods like **Boosting**.

Thank You!

Questions are welcome.